CSC3002
C/C++ Programming

List & Queue

Prof. Dr. Ruizhi Chen，SDS

Office: TxC 717

E-mail: chenruizhi@cuhk.edu.cn

1 The List class

2 The Queue class

3 The Priority Queue class

Methods in STL list class

fx Member functions

(constructor)	Construct list (public member funct
(destructor)	List destructor (public member fun
operator=	Assign content (public member fun

Iterators:

begin	Return iterator to beginning (pu
end	Return iterator to end (public mer
rbegin	Return reverse iterator to rever
rend	Return reverse iterator to rever
cbegin	Return const_iterator to beginr
cend	Return const_iterator to end (p
crbegin	Return const_reverse_iterator
crend	Return const_reverse_iterator

Capacity:

empty	Test whether container is emp
size	Return size (public member function
max_size	Return maximum size (public me

Element access:

front	Access first element (public mem
back	Access last element (public mem

Modifiers:

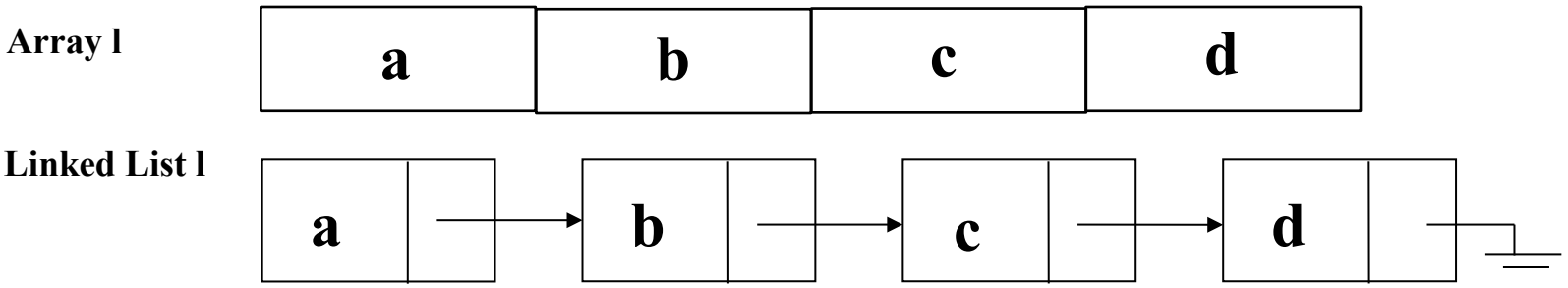
assign	Assign new content to container (public member function)
emplace_front	Construct and insert element at beginning (public member function)
push_front	Insert element at beginning (public member function)
pop_front	Delete first element (public member function)
emplace_back	Construct and insert element at the end (public member function)
push_back	Add element at the end (public member function)
pop_back	Delete last element (public member function)
emplace	Construct and insert element (public member function)
insert	Insert elements (public member function)
erase	Erase elements (public member function)
swap	Swap content (public member function)
resize	Change size (public member function)
clear	Clear content (public member function)

Operations:

splice	Transfer elements from list to list (public member function)
remove	Remove elements with specific value (public member function)
remove_if	Remove elements fulfilling condition (public member function template)
unique	Remove duplicate values (public member function)
merge	Merge sorted lists (public member function)
sort	Sort elements in container (public member function)
reverse	Reverse the order of elements (public member function)

Linked List Basics

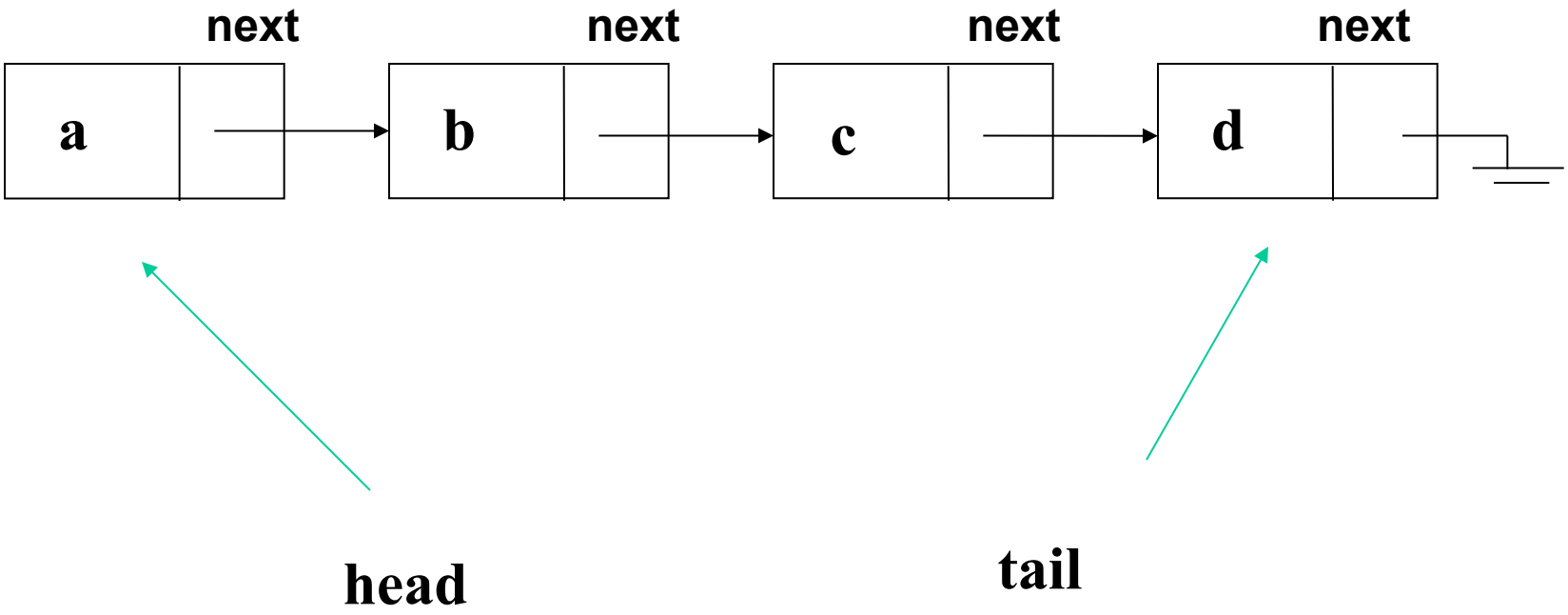
- Linked lists and arrays are similar since they both store collections of data.
- The *array's* features all follow from its strategy of allocating the memory for all its elements in one block of memory.
- *Linked lists* use an entirely different strategy: linked lists allocate memory for each element separately and only when necessary.



Linked List Basics

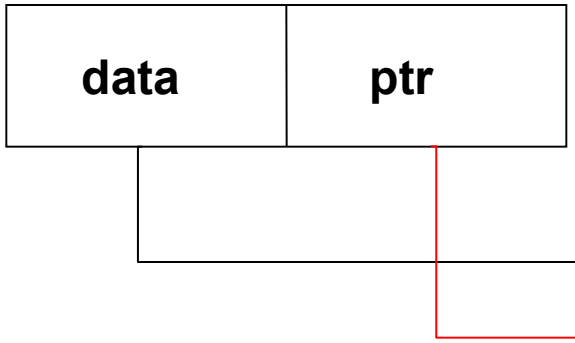
- Linked lists are used to store a collection of information (like arrays)
- A linked list is made of nodes that are pointing to each other
- We know the address of the first node (head) and the last node (tail)
- Other nodes are reached by following the “next” pointers

Linked List Basics



Linked List Basics

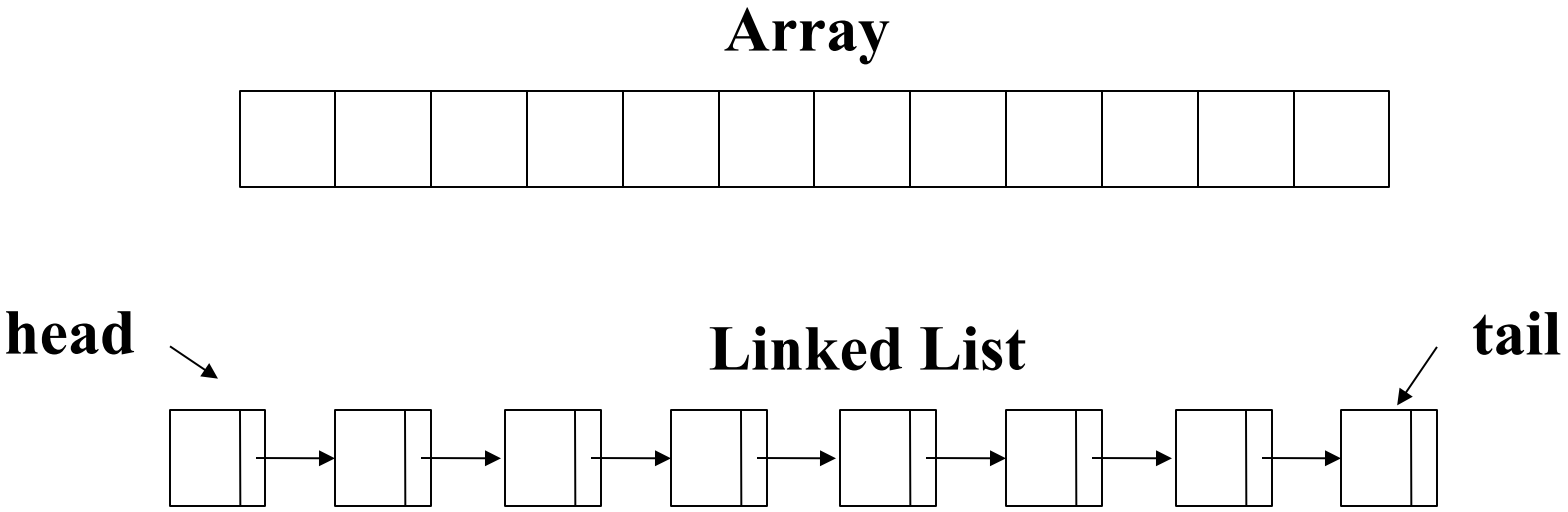
- **Each node has (at least) two fields:**
  - **Data**
  - **Pointer to the next node**




```
template <typename T>
class Node {
public:
  T element;
  Node *next;
};
```

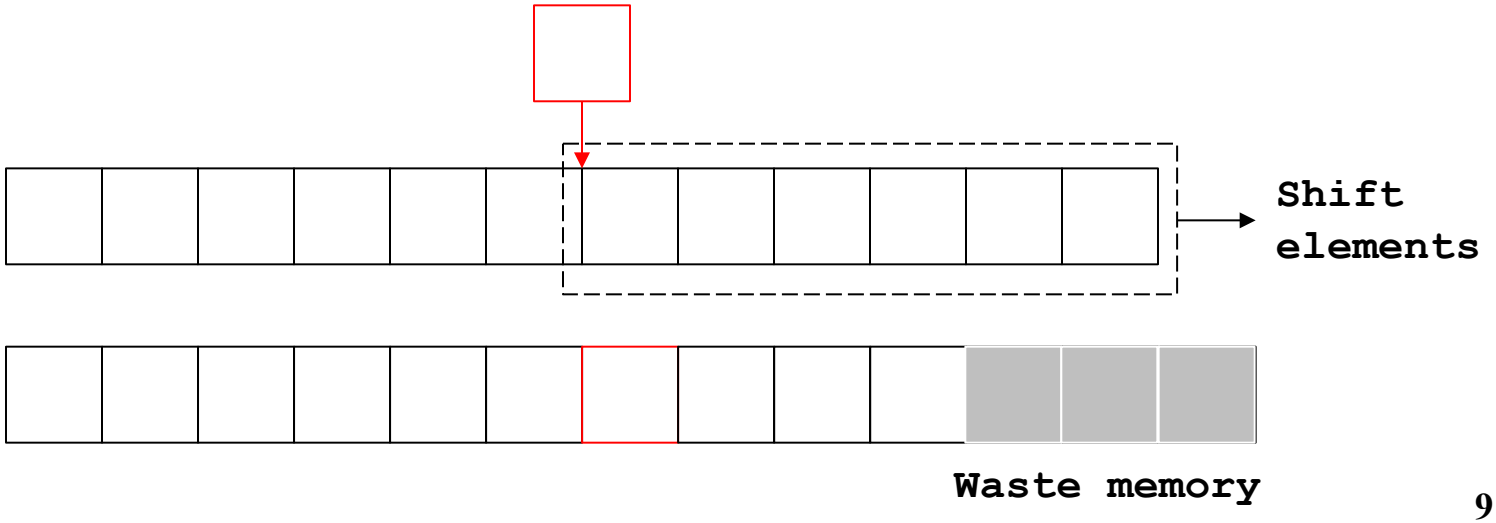
Linked List vs. Array

- In a linked list, nodes are not necessarily contiguous in memory (each node is allocated with a separate "new" call)
- Compare this to arrays which are contiguous



Linked List vs. Array

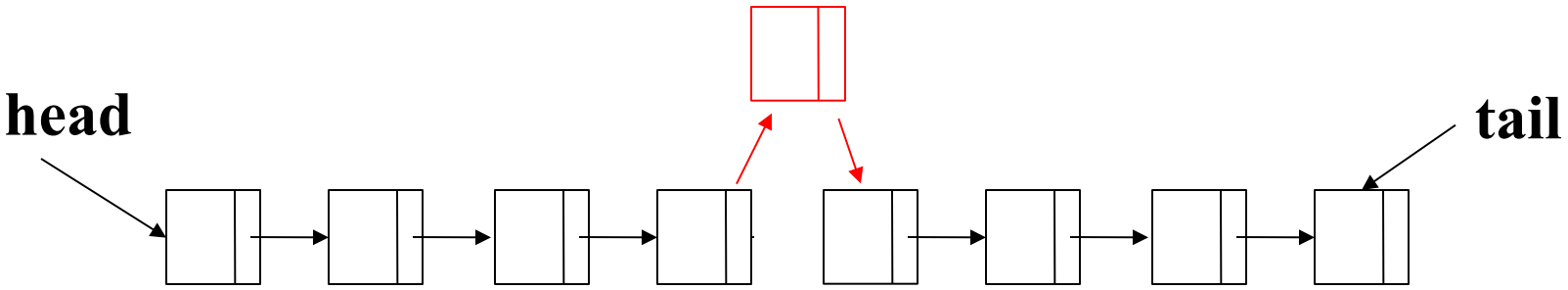
- **Advantages of Arrays**
  - Can directly select any element e.g. data[i]
  - No memory wasted for storing pointers
- **Disadvantages of Arrays:**
  - "Fixed" size (grow or shrink dynamically step-wise block-by-block)
  - Need to shift elements to insert an element to the middle
  - Memory wasted due to unused elements



Linked List vs. Array

- **Advantages of Linked Lists:**
  - **Dynamic size (can grow and shrink as needed)**
  - **No need to shift elements to insert into the middle**
  - **Size can exactly match the number of elements (no wasted memory)**

- **Disadvantages of Linked Lists**
  - **Cannot directly select any element (need to follow pointers)**
  - **Extra memory usage for storing pointers**



Linked List

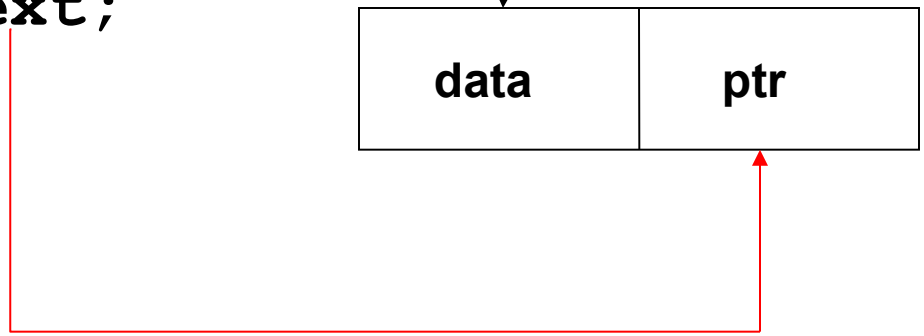
Linked List vs. Array

- **In general, we use linked lists if:**
  - **The number of elements that will be stored cannot be predicted at compile time**
  - **Elements may be inserted in the middle or deleted from the middle**
  - **We are less likely to make random access into the data structure like data[i] (because random access is expensive for linked lists, using array instead in this case)**

Linked List Implementation

- A linked list node can be represented as follows:

```
template <typename T>
class Node {
public:
   T element;
   Node *next;
};
```



Linked List Node Implementation

- **We can add a constructor to simplify creating a new node:**

```
template <typename T>
class Node {
public:
  Node(const T& e = T(), Node *n = NULL) :
      element(e), next(n) { }

  T element;
  Node *next;
};
```

Linked List Implementation

- **A linked list can be defined as follows**
- **Note that the constructor creates an empty list by making the head and tail points to NULL**

```
template <typename T>
class List
{
private:
        Node *head_ = nullptr;
        Node *tail_ = nullptr;
        std::size_t sz_ = 0; //number of elements in the list

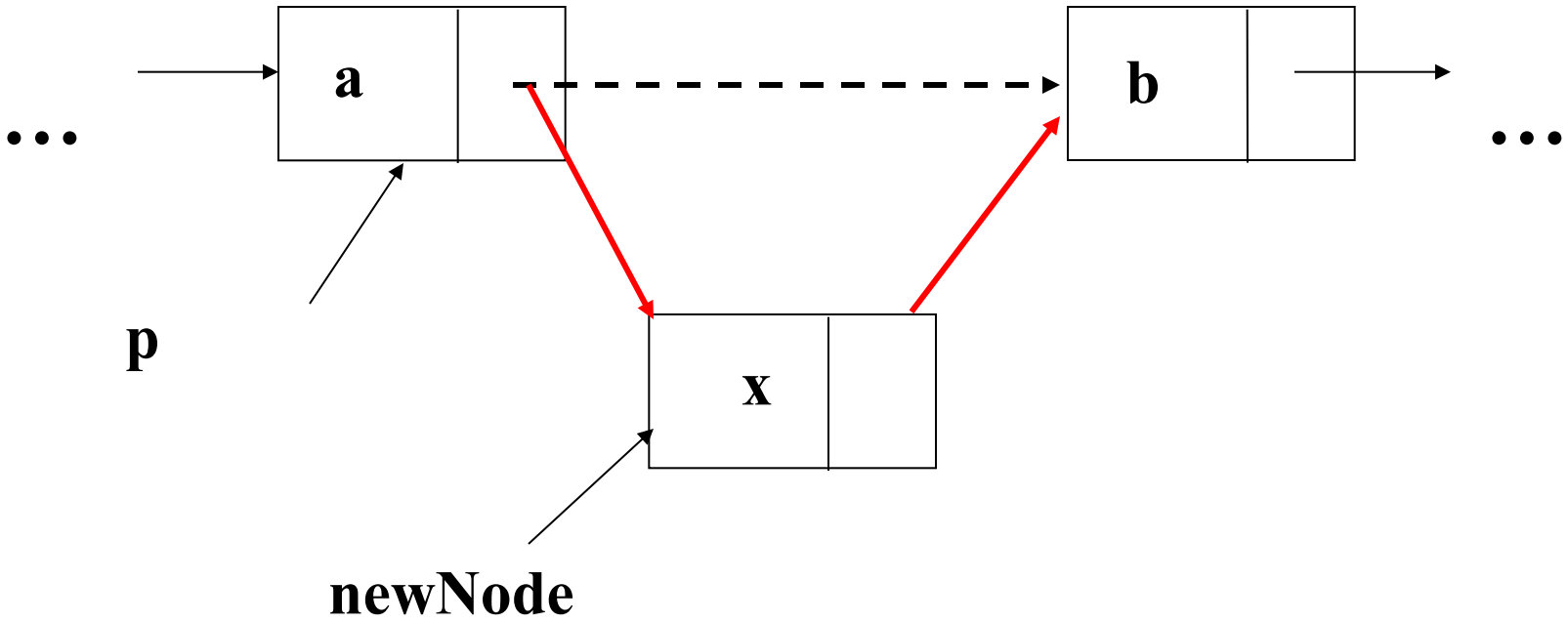
public:
   List() = default;
.
.//other functions.
.

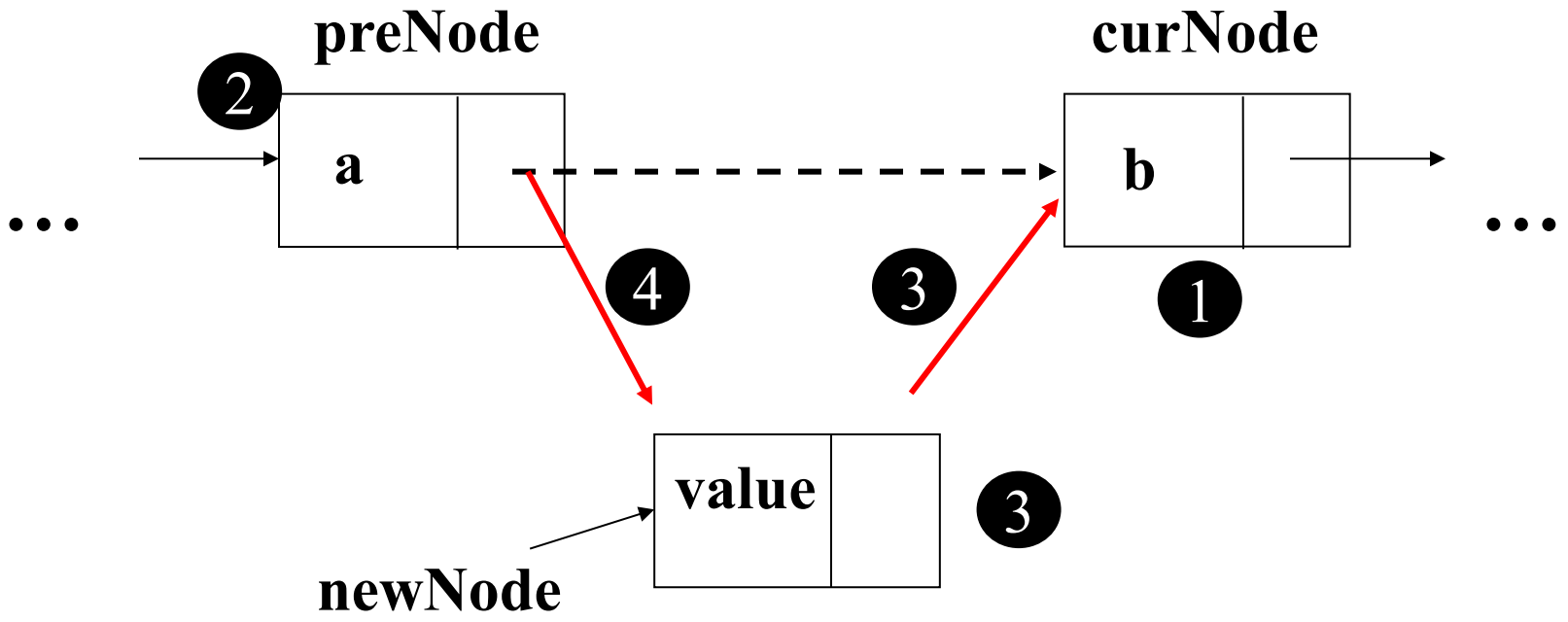
}
```

Basic Linked List Operations

- Insert a node
- Delete a node
- List Traversal
- Find a node
- … (much more functions)

Insertion in a linked list






```
void insert(iterator pos, const T &value)
{
.
.
```

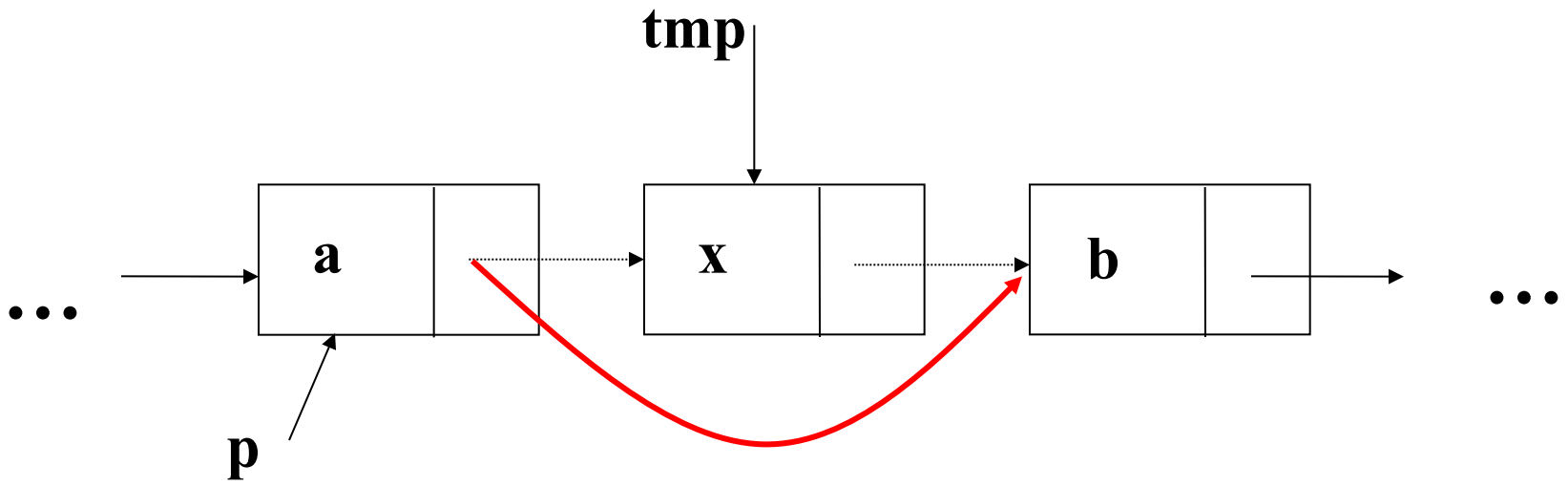
1 `Node *pCurrNode = pos.getNode(); //Get the node of the current pos`

2 `Node *pPrevNode = findPreviousNode(pos); // get previous node`

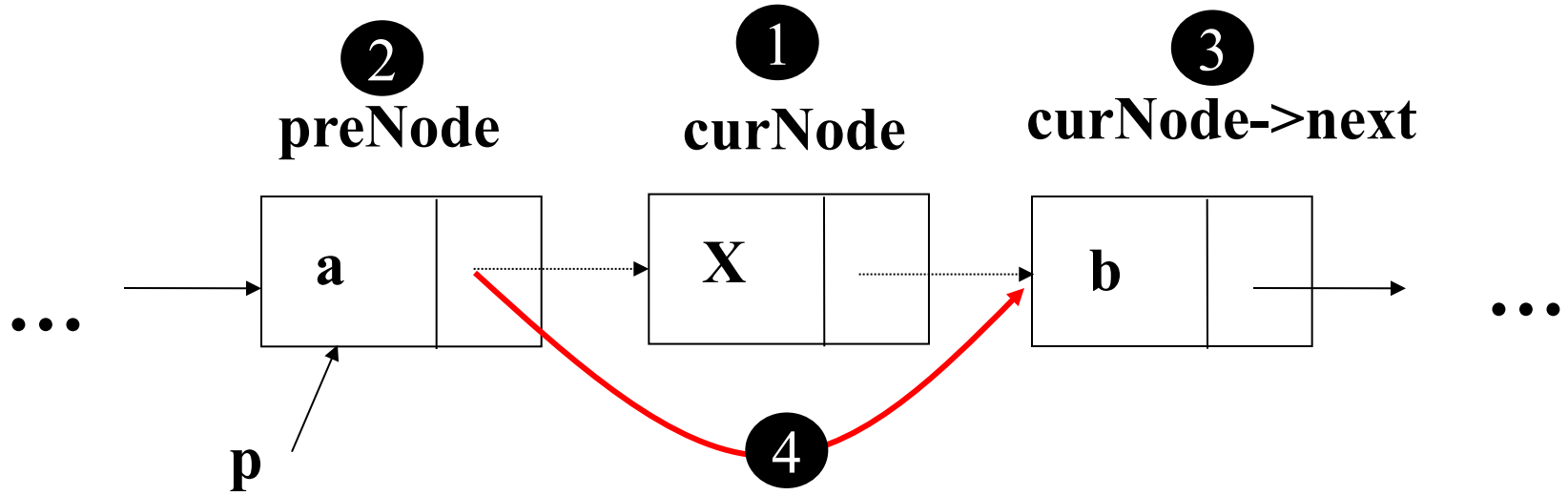
3 `Node *pNewNode = new Node(value, pCurrNode); // create a new now`

4
```
pPrevNode->next = pNewNode;
++sz_;
}
```

Removing a node from a linked list




```
Removing the node after p
```




```
void erase(iterator pos)
{
 ...
  Node *pCurrNode = pos.getNode(); // Get the node of the current pos

  Node *pPrevNode = findPreviousNode(pos); // get previous node

  Node *pNextNode = pCurrNode->next;

  pPrevNode->next = pNextNode;

  delete pCurrNode;

  --sz_;
}
```

The `list<type>` Class Operators

```
#include <list.h>

list<int> li;
list<int>::iterator it = li.begin();

li.push_back(10);

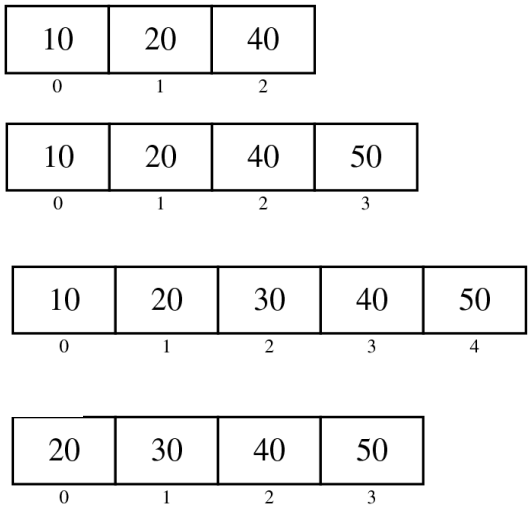
li.push_back(20);

li.push_back(40);

li.push_back(50);

advance(it,3);
li.insert(it, 30);

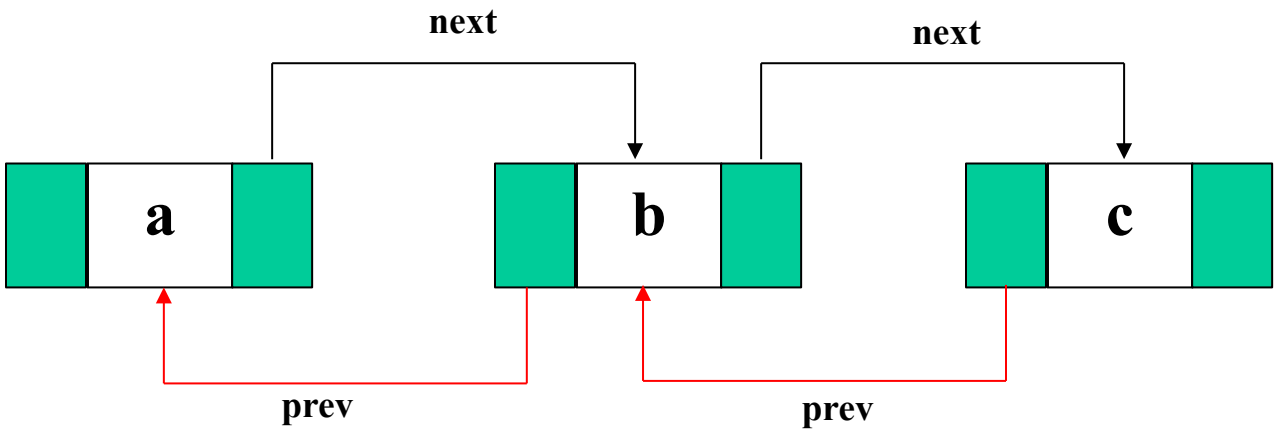
li.erase(li.begin());
```



The insert() function inserts new element before the element at the specified *position*.

MyList.h

Double-Linked List




```
template <typename T>
class Node {
public:
  Node(const T& e = T(), Node *n = NULL, Node *p = NULL) :
      element(e), next(n), prev(p) { }

  T element;
  Node *next;
  Node *prev;
};
```

Recognizing Palindrome

A string of characters that reads the same from left to right as it does from right to left.

LEVEL

Recognizing Palindromes

We can utilize a list to check the palindrome:

1. **Add each character to a list<char> object**
2. **Compare the chars from `front()` and `back()` of the list, if they are not the same, the word is not palindrome.**
3. **Pop both chars from top and back using `pop_front()`, `pop_back()`**
4. **Repeat steps 2,3, until the list is empty.**

`L E V E L`

```
bool isPalindrome(string &str)
{
        list<char> word;

        for (int i = 0; i < str.length(); i++) word.push_back(str[i]);

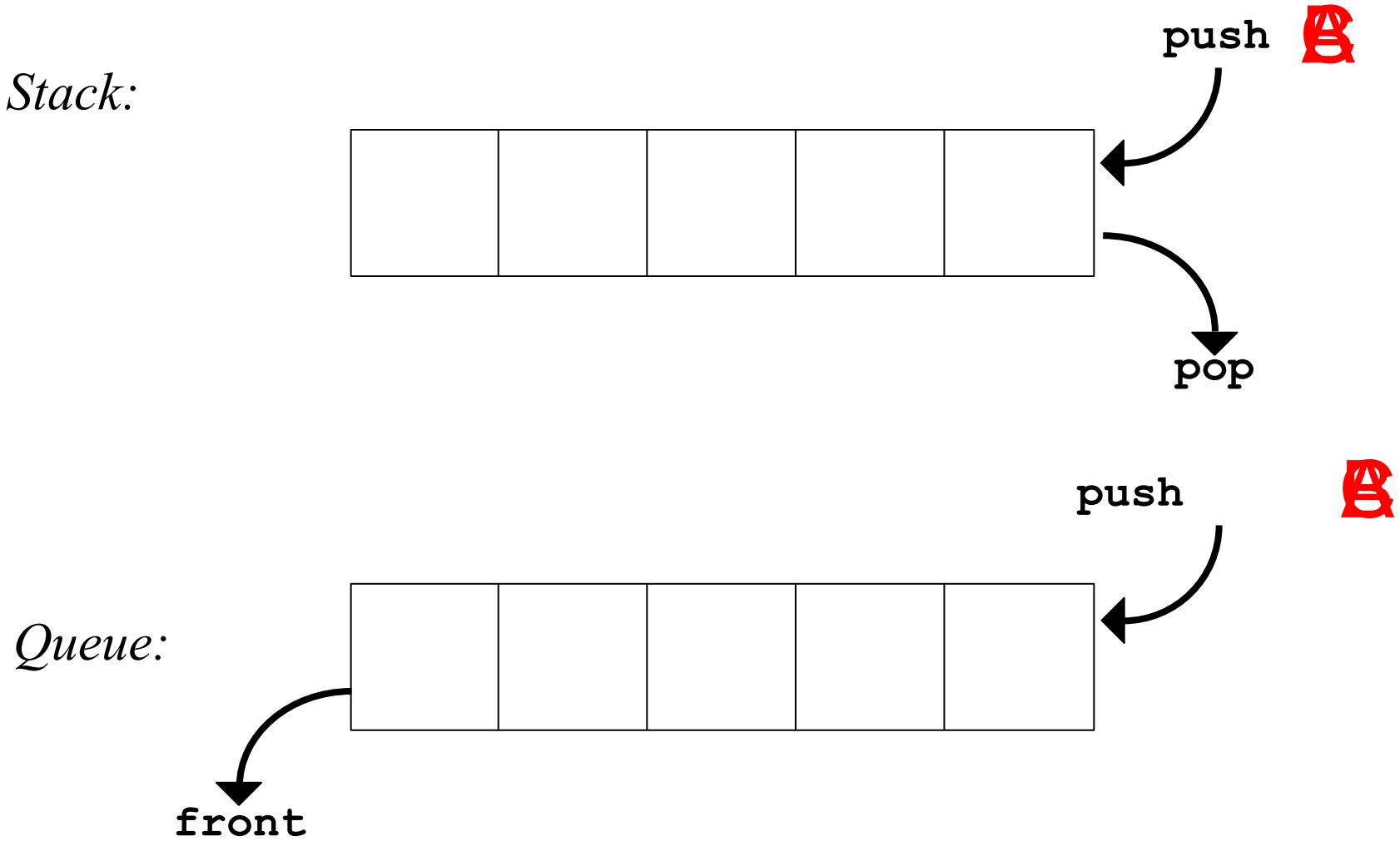
        while (word.size() > 1)
        {
                if (word.front() != word.back()) return false;
                word.pop_front();
                word.pop_back();
        }

   return true;
}
```

The **queue<*type*>** Class

- The ***LIFO*** discipline in a ***stack*** is useful in programming contexts because it reflects the operation of function calls.
- In real-world situations, however, our collective notion of fairness assigns some priority to being first. In programming, the usual phrasing of this ordering strategy is ***First In, First Out (FIFO)***.
- A data structure that stores items using a ***FIFO*** discipline is called a ***queue***. The fundamental operations on a queue, which are analogous to the **push** and **pop** operations for stacks, are called **push()** and **front()**.
- The **push()** operation adds a new element to the end of the queue, which is traditionally called its ***tail***.
- The **front()** operation removes the element at the beginning of the queue, which is called its ***head***.

Comparing Stacks and Queues



Methods in the STL **queue<*type*>** Class

fx Member functions

(constructor)	Construct queue (public member function)
empty	Test whether container is empty (public member function)
size	Return size (public member function)
front	Access next element (public member function)
back	Access last element (public member function)
push	Insert element (public member function)
emplace C++11	Construct and insert element (public member function)
pop	Remove next element (public member function)
swap C++11	Swap contents (public member function)

fx Non-member function overloads

relational operators	Relational operators for queue (function)
swap (queue) C++11	Exchange contents of queues (public member function)

fx Non-member class specializations

uses_allocator<queue> C++11	Uses allocator for queue (class template)

A simple implementation of a FIFO queue using linked list

MyQueue.h

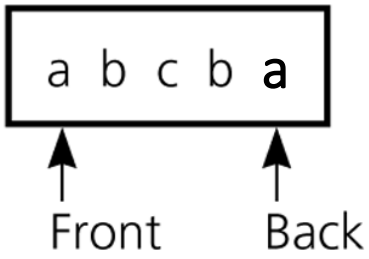
Recognizing Palindromes

- A palindrome
  - A string of characters that reads the same from left to right as it does from right to left
- To recognize a palindrome, a queue can be used in conjunction with a stack
  - A stack reverses the order of occurrences
  - A queue preserves the order of occurrences
- A non-recursive recognition algorithm for palindromes
  - As you traverse the character string from left to right, insert each character into both a queue and a stack
  - Compare the characters at the front of the queue and the top of the stack. (queue.front() = stack.top())

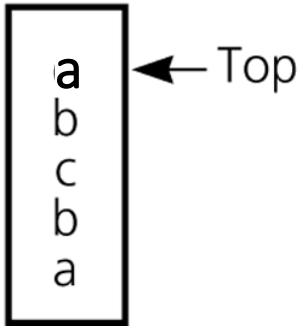
Recognizing Palindromes (cont.)

String: abcba

Queue:



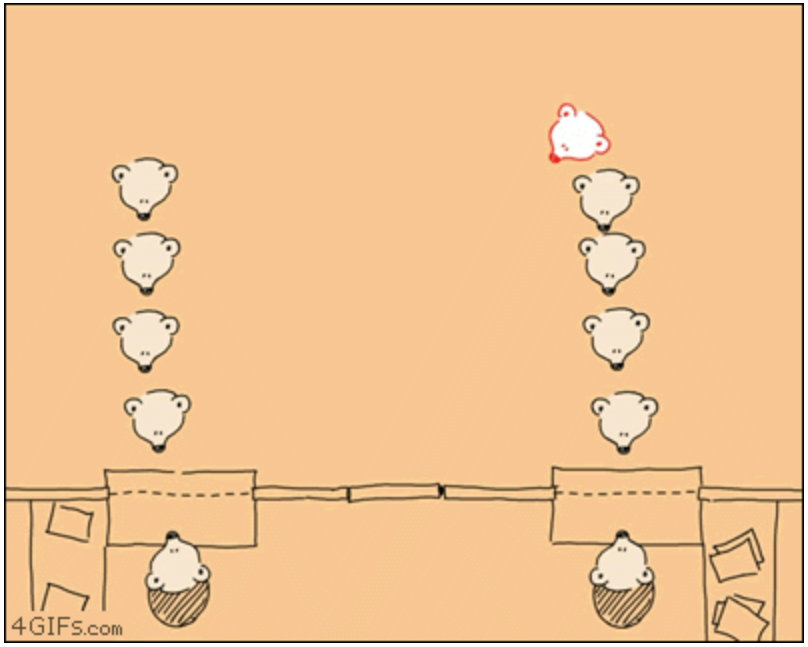
Stack:



The results of inserting a string into both a queue and a stack

Example: Simulate a checkout line

- One cashier is serving customers from a single queue.
- Customers arrive with a random probability and enter the queue at the end of the line.
- Whenever the cashier is free and someone is waiting in line, the cashier begins to serve that customer.
- After an appropriate service period, the cashier completes the transaction with the current customer, and is free to serve the next customer in the queue.



Example: Simulate a checkout line

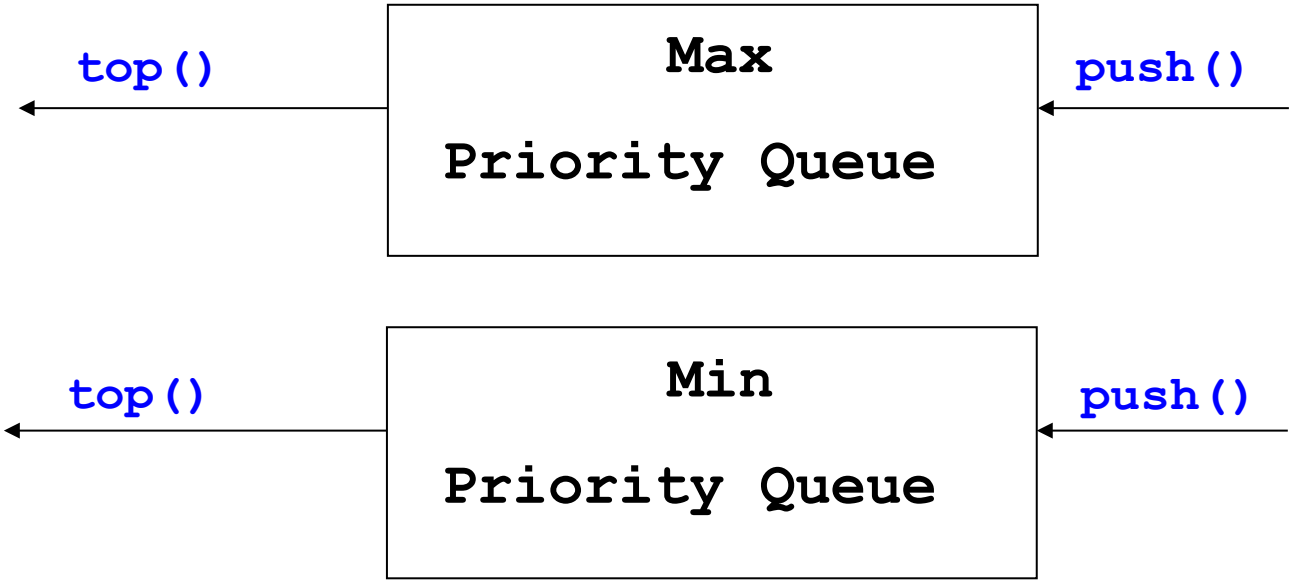
- The core of the simulation is a loop that runs for the number of seconds indicated by the parameter SIMULATION_TIME. In each second, the simulation performs the following operations:
  - Determine whether a new customer has arrived according to a certain probability ARRIVAL_PROBABILITY, and, if so, add that person to the queue.
  - If the cashier is busy, note that the cashier has spent another second with the current customer. Eventually, the required service time will be complete, which will free the cashier.
  - If the cashier is free, serve the next customer in the waiting line, for a certain amount of time, randomly chosen between MIN_SERVICE_TIME and MAX_SERVICE_TIME.

Priority Queues

- Many applications require that we process records with keys in order, but not necessarily in full sorted order.
- Often we collect a set of items and process the one with the current minimum value.
  - e.g. jobs sent to a printer,
  - Operating system job scheduler in a multi-user environment.
  - Simulation environments
- An appropriate data structure is called a *priority queue*.

Priority Queue

- A priority queue is a data structure that supports two basic operations: insert a new item and remove the minimum or maximum item.



Methods in the STL `priority_queue<type>` Class

fx Member functions

(constructor)	Construct priority queue (public member function)
empty	Test whether container is empty (public member function)
size	Return size (public member function)
top	Access top element (public member function)
push	Insert element (public member function)
emplace	Construct and insert element (public member function)
pop	Remove top element (public member function)
swap	Swap contents (public member function)

fx Non-member function overloads

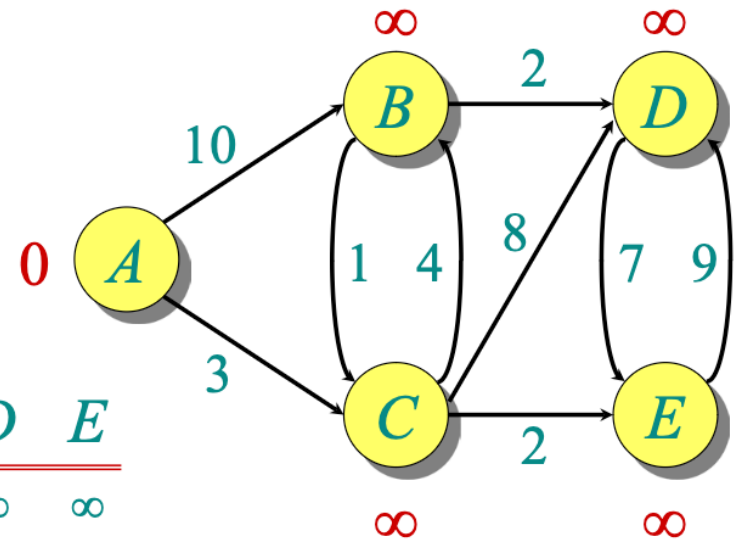
swap (queue)	Exchange contents of priority queues (public member function)

A simple implementation of a Priority Queue using vector

MyPriorityQueue.h

Finding the Shortest Path Using the Dijkstra's Algorithm

Initialize:



PriorityQueue

Q: *A* *B* *C* *D* *E*

0 ∞ ∞ ∞ ∞

S: {}

Details will be explained in the later lecture of Graph.

The End